Blind 75 Java Complete Guide (Problems 1–75)

1. Two Sum

Time: O(N) — Space: O(N)

Q: Find two indices such that elements sum to target.

Brute-force: Check all $O(N^2)$ pairs using nested loops ($O(N^2)$ time).

Optimal: Use HashMap to store complement (`target - num`) and index during iteration.

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int diff = target - nums[i];
        if (map.containsKey(diff)) return new
            int[]{map.get(diff), i};
        map.put(nums[i], i);
    }
    return new int[]{};
}
```

2. Best Time to Buy and Sell Stock

Time: O(N) — Space: O(1)

Q: Find maximum single transaction profit.

Brute-force: Calculate profit for all buy/sell pairs ($O(N^2)$ time).

Optimal: Track minimum price seen so far and calculate potential profit at each step.

```
public int maxProfit(int[] prices) {
    int min = Integer.MAX_VALUE, max = 0;
    for (int p : prices) {
        min = Math.min(min, p);
        max = Math.max(max, p - min);
    }
    return max;
}
```

3. Contains Duplicate

Time: O(N) — Space: O(N)

Q: Check if any value appears at least twice.

Brute-force: Compare every pair ($O(N^2)$) or sort array ($O(N \log N)$ time).

Optimal: Add items to HashSet; return true if element already exists.

```
public boolean containsDuplicate(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int n : nums) if (!set.add(n)) return true;
    return false;
}
```

4. Product of Array Except Self

Time: O(N) — Space: O(1)

Q: Compute array of products without using division.

Brute-force: Multiply all elements except i for each index ($O(N^2)$ time).

Optimal: Prefix pass stores cumulative products left to right; suffix pass multiplies right to left.

```
public int[] productExceptSelf(int[] nums) {
    int n = nums.length, res[] = new int[n];
    res[0] = 1;
    for (int i = 1; i < n; i++) res[i] = res[i-1] *
        nums[i-1];
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= right; right *= nums[i];
    }
    return res;
}
```

5. Maximum Subarray

Time: O(N) — Space: O(1)

Q: Find contiguous subarray with maximum sum.

Brute-force: Evaluate sum of all $O(N^2)$ possible subarrays ($O(N^3)$ or $O(N^2)$ time).

Optimal: Kadane's Algorithm: reset current sum to 0 if it drops below 0.

```
public int maxSubArray(int[] nums) {
    int max = nums[0], curr = 0;
    for (int n : nums) {
        curr = Math.max(n, curr + n);
        max = Math.max(max, curr);
    }
    return max;
}
```

6. Maximum Product Subarray

Time: O(N) — Space: O(1)

Q: Find contiguous subarray with maximum product.

Brute-force: Evaluate product of all $O(N^2)$ subarrays ($O(N^2)$ time).

Optimal: Maintain both running min and max products to handle negative sign flips.

```
public int maxProduct(int[] nums) {
    int res = nums[0], max = res, min = res;
    for (int i = 1; i < nums.length; i++) {
        if (nums[i] < 0) { int tmp = max; max = min;
            min = tmp; }
        max = Math.max(nums[i], max * nums[i]);
        min = Math.min(nums[i], min * nums[i]);
        res = Math.max(res, max);
    }
    return res;
}
```

7. Find Minimum in Rotated Sorted Array

Time: O(log N) — Space: O(1)

Q: Find minimum value in sorted array rotated k times.

Brute-force: Linear scan across all elements ($O(N)$ time).

Optimal: Binary search: if `mid > right`, minimum lies in right half; else left half.

```
public int findMin(int[] nums) {
    int l = 0, r = nums.length - 1;
    while (l < r) {
        int m = l + (r - l) / 2;
        if (nums[m] > nums[r]) l = m + 1;
        else r = m;
    }
    return nums[l];
}
```

8. Search in Rotated Sorted Array

Time: O(log N) — Space: O(1)

Q: Search target in rotated sorted array.

Brute-force: Linear search through elements ($O(N)$ time).

Optimal: Binary search: determine which half is sorted to decide search direction.

```
public int search(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (nums[m] == target) return m;
        if (nums[l] <= nums[m]) {
            if (target >= nums[l] && target <
                nums[m]) r = m - 1;
            else l = m + 1;
        } else {
            if (target > nums[m] && target <=
                nums[r]) l = m + 1;
            else r = m - 1;
        }
    }
    return -1;
}
```

9. 3Sum

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums); List<List<Integer>> res = new
        ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue;
        int l = i + 1, r = nums.length - 1;
        while (l < r) {
            int sum = nums[i] + nums[l] + nums[r];
            if (sum == 0) {
                res.add(Arrays.asList(nums[i],
                    nums[l++], nums[r--]));
                while (l < r && nums[l] ==
                    nums[l-1]) l++;
            } else if (sum < 0) l++; else r--;
        }
    }
    return res;
}
```

10. Container With Most Water

Time: O(N) — Space: O(1)

Q: Find two lines forming container holding maximum water.

Brute-force: Calculate area for all $O(N^2)$ line pairs ($O(N^2)$ time).

Optimal: Two pointers at ends; move pointer with smaller height inward.

```
public int maxArea(int[] height) {
    int l = 0, r = height.length - 1, max = 0;
    while (l < r) {
        max = Math.max(max, Math.min(height[l],
            height[r]) * (r - l));
        if (height[l] < height[r]) l++; else r--;
    }
    return max;
}
```

11. Sum of Two Integers

Time: O(1) — Space: O(1)

Q: Add two integers without using $+$ or $-$ operators.

Brute-force: Simulate addition with increment operations ($O(N)$ time).

Optimal: Bit manipulation: XOR (`^`) handles addition without carry; AND (`&`) left-shifted computes carry.

```
public int getSum(int a, int b) {
    while (b != 0) {
        int carry = (a & b) << 1;
        a = a ^ b;
        b = carry;
    }
    return a;
}
```

12. Number of 1 Bits

Time: O(1) — Space: O(1)

Q: Count set bits (Hamming weight) of an integer.

Brute-force: Loop through all 32 bit positions checking mask ($O(32)$ steps).

Optimal: Brian Kernighan's Algorithm: `n &= (n - 1)` clears lowest set bit in $O(setbits)$ steps.

```
public int hammingWeight(int n) {
    int count = 0;
    while (n != 0) { n &= (n - 1); count++; }
    return count;
}
```

13. Counting Bits

Time: O(N) — Space: O(1)

Q: Return array of set bit counts from 0 to n.

Brute-force: Run `hammingWeight` on each number independently ($O(N \log N)$ time).

Optimal: Dynamic programming: `dp[i] = dp[i >> 1] + (i & 1)`.

```
public int[] countBits(int n) {
    int[] res = new int[n + 1];
    for (int i = 1; i <= n; i++) res[i] = res[i >>
        1] + (i & 1);
    return res;
}
```

14. Missing Number

Time: O(N) — Space: O(1)

Q: Find missing number in range $[0, n]$.

Brute-force: Sort array and check index mismatch ($O(N \log N)$ time) or check HashSet ($O(N)$ space).

Optimal: XOR all indices and elements; matching pairs cancel leaving missing number.

```
public int missingNumber(int[] nums) {
    int res = nums.length;
    for (int i = 0; i < nums.length; i++) res ^= i ^
        nums[i];
    return res;
}
```

15. Reverse Bits

Time: O(1) — Space: O(1)

Q: Reverse 32-bit unsigned integer bits.

Brute-force: Convert to binary string, reverse string, parse back ($O(32)$ space/string operations).

Optimal: Shift result left and insert rightmost bit of input using bitwise shifts.

```
public int reverseBits(int n) {
    int res = 0;
    for (int i = 0; i < 32; i++) {
        res = (res << 1) | (n & 1);
        n >>= 1;
    }
    return res;
}
```

16. Climbing Stairs

Time: O(N) — Space: O(1)

Q: Count ways to reach top taking 1 or 2 steps.

Brute-force: Naive recursion $f(n) = f(n-1) + f(n-2)$ ($O(2^N)$ time).

Optimal: Fibonacci dynamic programming maintaining previous two step combinations.

```
public int climbStairs(int n) {
    int a = 1, b = 1;
    for (int i = 0; i < n - 1; i++) {
        int tmp = a + b; a = b; b = tmp;
    }
    return b;
}
```

17. Coin Change

Time: O(N * amount) — Space: O(amount)

Q: Minimum coins needed to make given amount.

Brute-force: Brute-force recursive search over all coin subsets ($O(amount^{coins})$).

Optimal: Bottom-up DP array where `dp[i] = min(dp[i], dp[i - coin] + 1)`.

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int c : coins) if (i >= c) dp[i] =
            Math.min(dp[i], dp[i - c] + 1);
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

18. Longest Increasing Subsequence

Time: O(N log N) — Space: O(N)

Q: Find length of longest strictly increasing subsequence.

Brute-force: Standard $O(N^2)$ dynamic programming comparing all previous elements.

Optimal: Maintain tail array using Binary Search (`Arrays.binarySearch`) to build optimal tails.

```
public int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length]; int len = 0;
    for (int x : nums) {
        int i = Arrays.binarySearch(tails, 0, len,
            x);
        if (i < 0) i = -(i + 1);
        tails[i] = x;
        if (i == len) len++;
    }
    return len;
}
```

19. Longest Common Subsequence

Time: O(M * N) — Space: O(M * N)

Q: Find length of longest common subsequence of two strings.

Brute-force: Generate all sub-sequences for both strings ($O(2^{M+N})$ time).

Optimal: 2D DP matrix matching characters: `dp[i][j] = dp[i-1][j-1] + 1` on match.

```
public int longestCommonSubsequence(String text1,
    String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i-1) ==
                text2.charAt(j-1)) dp[i][j] =
                dp[i-1][j-1] + 1;
            else dp[i][j] = Math.max(dp[i-1][j],
                dp[i][j-1]);
        }
    }
    return dp[m][n];
}
```

20. Word Break Problem

```
public boolean wordBreak(String s, List<String>
    wordDict) {
    Set<String> set = new HashSet<>(wordDict);
    boolean[] dp = new boolean[s.length() + 1];
        dp[0] = true;
    for (int i = 1; i <= s.length(); i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && set.contains(s.substring(j,
                i))) {
                dp[i] = true; break;
            }
        }
    }
```

```
    return dp[s.length()];
}
```

21. Combination Sum IV

Time: O(N * target) — Space: O(target)

Q: Find number of target combination permutations.

Brute-force: Recursively search combinations without memoization ($O(N^{target})$).

Optimal: 1D DP array where `dp[i]` aggregates combinations from `dp[i - num]`.

```
public int combinationSum4(int[] nums, int target) {
    int[] dp = new int[target + 1]; dp[0] = 1;
    for (int i = 1; i <= target; i++) {
        for (int num : nums) if (i >= num) dp[i] +=
            dp[i - num];
    }
    return dp[target];
}
```

22. House Robber

Time: O(N) — Space: O(1)

Q: Max value stolen without robbing adjacent houses.

Brute-force: Try all subset choices avoiding adjacent picks recursively ($O(2^N)$ time).

Optimal: DP tracking max money up to current house: `curr = max(prev1, prev2 + val)`.

```
public int rob(int[] nums) {
    int p1 = 0, p2 = 0;
    for (int n : nums) {
        int tmp = Math.max(p1, p2 + n); p2 = p1; p1
            = tmp;
    }
    return p1;
}
```

23. House Robber II

Time: O(N) — Space: O(1)

Q: Max value stolen where houses are arranged in a circle.

Brute-force: Evaluate all subsets under circular constraints recursively ($O(2^N)$ time).

Optimal: Run standard House Robber twice: once on `0..N-2` and once on `1..N-1`.

```
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    return Math.max(helper(nums, 0, nums.length -
        2), helper(nums, 1, nums.length - 1));
}
private int helper(int[] nums, int s, int e) {
    int p1 = 0, p2 = 0;
    for (int i = s; i <= e; i++) {
        int tmp = Math.max(p1, p2 + nums[i]); p2 =
            p1; p1 = tmp;
    }
    return p1;
}
```

24. Decode Ways

Time: O(N) — Space: O(1)

Q: Count ways to decode a digit string ($A = 1, \ldots, Z = 26$).

Brute-force: Recursively branch on single vs double digit choices ($O(2^N)$ time).

Optimal: DP tracking valid single-digit ($1 - 9$) and double-digit ($10 - 26$) configurations.

```
public int numDecodings(String s) {
    if (s.charAt(0) == '0') return 0;
    int p1 = 1, p2 = 1;
    for (int i = 1; i < s.length(); i++) {
        int curr = 0;
        int v1 = s.charAt(i) - '0', v2 =
            Integer.parseInt(s.substring(i - 1, i +
            1));
        if (v1 != 0) curr += p1;
        if (v2 >= 10 && v2 <= 26) curr += p2;
        p2 = p1; p1 = curr;
    }
    return p1;
}
```

25. Unique Paths

Time: O(M * N) — Space: O(N)

Q: Find unique paths from top-left to bottom-right of grid.

Brute-force: DFS explore all downward and rightward branches ($O(2^{M+N})$ time).

Optimal: 1D dynamic programming array tracking path sums row by row.

```
public int uniquePaths(int m, int n) {
    int[] dp = new int[n]; Arrays.fill(dp, 1);
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) dp[j] += dp[j -
            1];
    }
    return dp[n - 1];
}
```

26. Jump Game

Time: O(N) — Space: O(1)

Q: Determine if you can reach the final index.

Brute-force: Backtracking: explore all possible jump distances from each index ($O(2^N)$ time).

Optimal: Greedy: track maximum reachable index linearly; return false if current index exceeds it.

```
public boolean canJump(int[] nums) {
    int max = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > max) return false;
        max = Math.max(max, i + nums[i]);
    }
    return true;
}
```

27. Clone Graph

Time: O(V + E) — Space: O(V)

Q: Deep copy an undirected connected graph.

Brute-force: DFS copy nodes without memoization causing infinite loops ($O(\infty)$ time).

Optimal: DFS/BFS using HashMap to map original nodes to their cloned counterparts.

```
Map<Node, Node> map = new HashMap<>();
public Node cloneGraph(Node node) {
    if (node == null) return null;
    if (map.containsKey(node)) return map.get(node);
    Node copy = new Node(node.val);
    map.put(node, copy);
    for (Node n : node.neighbors)
        copy.neighbors.add(cloneGraph(n));
    return copy;
}
```

28. Course Schedule

Time: O(V + E) — Space: O(V + E)

Q: Determine if all courses can be finished (detect cycle).

Brute-force: Check every course path independently using DFS ($O(V!)$ time).

Optimal: Kahn's Algorithm (BFS topological sort) tracking node in-degrees.

```
public boolean canFinish(int numCourses, int[][]
    prerequisites) {
    List<Integer>[] adj = new ArrayList[numCourses];
    int[] in = new int[numCourses];
    for (int i = 0; i < numCourses; i++) adj[i] =
        new ArrayList<>();
    for (int[] p : prerequisites) {
        adj[p[1]].add(p[0]); in[p[0]]++; }
    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) if (in[i]
        == 0) q.add(i);
    int count = 0;
    while (!q.isEmpty()) {
        int curr = q.poll(); count++;
        for (int next : adj[curr]) if (--in[next] ==
            0) q.add(next);
    }
    return count == numCourses;
}
```

29. Number of Islands

Time: O(M * N) — Space: O(M * N)

Q: Count connected components of '1's in 2D grid.

Brute-force: Explore grid paths without marking visited cells ($O(\infty)$ depth recursion).

Optimal: Traverse grid; trigger DFS/BFS on encountering '1' to sink connected component to '0'.

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') { count++;
                dfs(grid, r, c); }
        }
    }
    return count;
}
private void dfs(char[][] g, int r, int c) {
    if (r<0 || c<0 || r>=g.length || c>=g[0].length
        || g[r][c] != '1') return;
    g[r][c] = '0';
    dfs(g, r+1, c); dfs(g, r-1, c); dfs(g, r, c+1);
        dfs(g, r, c-1);
}
```

30. Pacific Atlantic Water Flow

Time: O(M * N) — Space: O(M * N)

Q: Find cells where water flows to both Pacific and Atlantic.

Brute-force: Run ocean-reachability search from every grid cell ($O((M \cdot N)^2)$ time).

Optimal: DFS inward starting from Pacific and Atlantic ocean edges separately; intersect results.

```
public List<List<Integer>> pacificAtlantic(int[][]
    heights) {
    int r = heights.length, c = heights[0].length;
    boolean[][] pac = new boolean[r][c], atl = new
        boolean[r][c];
    for (int i = 0; i < r; i++) { dfs(heights, pac,
        i, 0); dfs(heights, atl, i, c - 1); }
    for (int j = 0; j < c; j++) { dfs(heights, pac,
        0, j); dfs(heights, atl, r - 1, j); }
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            if (pac[i][j] && atl[i][j])
                res.add(Arrays.asList(i, j));
    return res;
}
private void dfs(int[][] h, boolean[][] ocean, int
    r, int c) {
    ocean[r][c] = true;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    for (int[] d : dirs) {
        int nr = r + d[0], nc = c + d[1];
        if (nr>=0 && nc>=0 && nr<h.length &&
            nc<h[0].length && !ocean[nr][nc] &&
            h[nr][nc]>=h[r][c])
            dfs(h, ocean, nr, nc);
    }
}
```

31. Longest Consecutive Sequence

Time: O(N) — Space: O(N)

Q: Find length of longest consecutive elements sequence.

Brute-force: Sort array and scan linearly ($O(N \log N)$ time).

Optimal: Add all numbers to HashSet; only start counting sequence lengths when `!set.contains(n - 1)`.

```
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int n : nums) set.add(n);
    int max = 0;
    for (int n : set) {
        if (!set.contains(n - 1)) {
            int curr = n, len = 1;
            while (set.contains(curr + 1)) { curr++;
                len++; }
            max = Math.max(max, len);
        }
    }
    return max;
}
```

32. Alien Dictionary

Time: O(C) — Space: O(1)

Q: Find character order from alien sorted dictionary.

Brute-force: Permute all 26 character ordering possibilities ($O(26!)$ time).

Optimal: Build directed graph based on lexicographical character mismatches; run Topological Sort.

```
public String alienOrder(String[] words) {
    Map<Character, Set<Character>> adj = new
        HashMap<>();
    Map<Character, Integer> in = new HashMap<>();
    for (String w : words) for (char c :
        w.toCharArray()) { adj.put(c, new
        HashSet<>()); in.put(c, 0); }
    for (int i = 0; i < words.length - 1; i++) {
        String w1 = words[i], w2 = words[i+1];
        if (w1.length() > w2.length() &&
            w1.startsWith(w2)) return "";
        for (int j = 0; j < Math.min(w1.length(),
            w2.length()); j++) {
            char c1 = w1.charAt(j), c2 =
                w2.charAt(j);
            if (c1 != c2) {
                if (!adj.get(c1).contains(c2)) {
                    adj.get(c1).add(c2); in.put(c2,
                    in.get(c2) + 1); }
                break;
            }
        }
    }
    Queue<Character> q = new LinkedList<>();
    for (char c : in.keySet()) if (in.get(c) == 0)
        q.add(c);
    StringBuilder sb = new StringBuilder();
    while (!q.isEmpty()) {
        char c = q.poll(); sb.append(c);
        for (char next : adj.get(c)) { in.put(next,
            in.get(next) - 1); if (in.get(next) ==
            0) q.add(next); }
    }
    return sb.length() == in.size() ? sb.toString()
        : "";
}
```

33. Graph Valid Tree

Time: O(V + E) — Space: O(V)

Q: Check if graph edges form valid tree.

Brute-force: Run cycle check DFS without enforcing $E = V - 1$ edge count check ($O(V \cdot E)$).

Optimal: Valid tree iff $E = V - 1$ and all nodes are fully connected via Union-Find or BFS/DFS.

```
public boolean validTree(int n, int[][] edges) {
    if (edges.length != n - 1) return false;
    int[] parent = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
    for (int[] e : edges) {
        int r1 = find(parent, e[0]), r2 =
            find(parent, e[1]);
        if (r1 == r2) return false;
        parent[r1] = r2;
    }
    return true;
}
private int find(int[] p, int i) {
    if (p[i] == i) return i;
    return p[i] = find(p, p[p[i]]);
}
```

34. Number of Connected Components

Time: O(V + E) — Space: O(V)

Q: Find total connected components in undirected graph.

Brute-force: DFS traversal storing visited nodes in a list with linear lookups ($O(V^2)$ time).

Optimal: Union-Find initialized with N components; decrement count upon successful union.

```
public int countComponents(int n, int[][] edges) {
    int[] p = new int[n];
    for (int i = 0; i < n; i++) p[i] = i;
    int count = n;
    for (int[] e : edges) {
        int r1 = find(p, e[0]), r2 = find(p, e[1]);
        if (r1 != r2) { p[r1] = r2; count--; }
    }
    return count;
}
private int find(int[] p, int i) { return p[i] == i
    ? i : (p[i] = find(p, p[i])); }
```

35. Insert Interval

Time: O(N) — Space: O(N)

Q: Insert and merge new interval into sorted non-overlapping intervals.

Brute-force: Insert new interval, then re-sort full array ($O(N \log N)$ time).

Optimal: Single pass: add non-overlapping left intervals, merge overlapping, add non-overlapping right.

```
public int[][] insert(int[][] intervals, int[]
    newInterval) {
    List<int[]> res = new ArrayList<>();
    int i = 0, n = intervals.length;
    while (i < n && intervals[i][1] <
        newInterval[0]) res.add(intervals[i++]);
    while (i < n && intervals[i][0] <=
        newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0],
            intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1],
            intervals[i][1]);
        i++;
    }
    res.add(newInterval);
    while (i < n) res.add(intervals[i++]);
    return res.toArray(new int[res.size()][]);
}
```

36. Merge Intervals

Time: O(N log N) — Space: O(N)

Q: Merge all overlapping intervals.

Brute-force: Compare every interval against all others iteratively ($O(N^2)$ time).

Optimal: Sort intervals by start time; iterate through and merge overlapping intervals into output list.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) ->
        Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    int[] curr = intervals[0];
    res.add(curr);
    for (int[] next : intervals) {
        if (next[0] <= curr[1]) curr[1] =
            Math.max(curr[1], next[1]);
        else { curr = next; res.add(curr); }
    }
    return res.toArray(new int[res.size()][]);
}
```

37. Non-overlapping Intervals

Time: O(N log N) — Space: O(1)

Q: Minimum removals to make remaining intervals non-overlapping.

Brute-force: Backtracking: compute subsets of non-overlapping intervals ($O(2^N)$ time).

Optimal: Greedy strategy: sort by end times; retain interval with earliest end time to reduce overlaps.

```
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) ->
        Integer.compare(a[1], b[1]));
    int count = 0, end = intervals[0][1];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < end) count++;
        else end = intervals[i][1];
    }
    return count;
}
```

38. Meeting Rooms

Time: O(N log N) — Space: O(1)

Q: Determine if a person can attend all meetings.

Brute-force: Compare every pair of meeting times for overlap ($O(N^2)$ time).

Optimal: Sort intervals by start time; check if any interval starts before previous ends.

```
public boolean canAttendMeetings(int[][] intervals) {
    Arrays.sort(intervals, (a, b) ->
        Integer.compare(a[0], b[0]));
    for (int i = 1; i < intervals.length; i++)
        if (intervals[i][0] < intervals[i-1][1])
            return false;
    return true;
}
```

39. Meeting Rooms II

Time: O(N log N) — Space: O(N)

Q: Find minimum meeting rooms required.

Brute-force: Track rooms linearly by checking all active meetings for each new assignment ($O(N^2)$ time).

Optimal: Sort start and end times independently; use two pointers to calculate active overlaps.

```
public int minMeetingRooms(int[][] intervals) {
    int[] starts = new int[intervals.length], ends =
        new int[intervals.length];
    for (int i = 0; i < intervals.length; i++) {
        starts[i] = intervals[i][0]; ends[i] =
        intervals[i][1]; }
    Arrays.sort(starts); Arrays.sort(ends);
    int rooms = 0, endIdx = 0;
    for (int i = 0; i < starts.length; i++) {
        if (starts[i] < ends[endIdx]) rooms++;
        else endIdx++;
    }
    return rooms;
}
```

40. Rotate Image

```
public void rotate(int[][] matrix) {
    int n = matrix.length;
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++) {
            int tmp = matrix[i][j]; matrix[i][j] =
                matrix[j][i]; matrix[j][i] = tmp;
        }
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n / 2; j++) {
            int tmp = matrix[i][j]; matrix[i][j] =
                matrix[i][n - 1 - j]; matrix[i][n -
                1 - j] = tmp;
        }
}
```

41. Spiral Matrix

Time: O(M * N) — Space: O(1)

Q: Return elements in spiral order.

Brute-force: Track grid direction vectors and vis arrays ($O(M \cdot N)$ extra space).

Optimal: Maintain 4 boundary pointers (`top`, `bottom`, `left`, `right`) and increment inwards.

```
public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> res = new ArrayList<>();
    int top = 0, bottom = matrix.length - 1, left =
        0, right = matrix[0].length - 1;
    while (left <= right && top <= bottom) {
        for (int j = left; j <= right; j++)
            res.add(matrix[top][j]); top++;
        for (int i = top; i <= bottom; i++)
            res.add(matrix[i][right]); right--;
        if (top <= bottom) { for (int j = right; j
            >= left; j--)
            res.add(matrix[bottom][j]); bottom--; }
        if (left <= right) { for (int i = bottom; i
            >= top; i--) res.add(matrix[i][left]);
            left++; }
    }
    return res;
}
```

42. Set Matrix Zeroes

Time: O(M * N) — Space: O(1)

Q: If element is 0, set its entire row and column to 0.

Brute-force: Create duplicate $M \times N$ matrix to mark zeroes ($O(M \cdot N)$ space).

Optimal: Use first row and column as zero flags; use extra variable for matrix cell $[0][0]$.

```
public void setZeroes(int[][] matrix) {
    boolean col0 = false;
    int r = matrix.length, c = matrix[0].length;
    for (int i = 0; i < r; i++) {
        if (matrix[i][0] == 0) col0 = true;
        for (int j = 1; j < c; j++)
            if (matrix[i][j] == 0) matrix[i][0] =
                matrix[0][j] = 0;
    }
    for (int i = r - 1; i >= 0; i--) {
        for (int j = c - 1; j >= 1; j--)
            if (matrix[i][0] == 0 || matrix[0][j] ==
                0) matrix[i][j] = 0;
        if (col0) matrix[i][0] = 0;
    }
}
```

43. Reverse Linked List

Time: O(N) — Space: O(1)

Q: Reverse a singly linked list.

Brute-force: Store values in array and create new list in reverse ($O(N)$ space).

Optimal: Iterative pointer modification: maintain `prev`, `curr`, `next` pointers.

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next; curr.next = prev;
            prev = curr; curr = next;
    }
    return prev;
}
```

44. Linked List Cycle

Time: O(N) — Space: O(1)

Q: Determine if linked list contains cycle.

Brute-force: Store visited node references in HashSet ($O(N)$ space).

Optimal: Floyd's Cycle Detection (Slow and Fast pointers); return true if pointers collide.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

45. Merge Two Sorted Lists

Time: O(N + M) — Space: O(1)

Q: Merge two sorted lists into one.

Brute-force: Append elements into array, sort array, build new list ($O((N+M)\log(N+M))$ time).

Optimal: Use dummy node; compare heads of both lists and advance pointers iteratively.

```
public ListNode mergeTwoLists(ListNode l1, ListNode
    l2) {
```

```
    ListNode dummy = new ListNode(0), curr = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) { curr.next = l1; l1 =
            l1.next; }
        else { curr.next = l2; l2 = l2.next; }
        curr = curr.next;
    }
    curr.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

46. Remove Nth Node From End of List

Time: O(N) — Space: O(1)

Q: Remove n-th node from list end.

Brute-force: First pass counts length L, second pass removes node at $L - n$ (2 passes).

Optimal: Two pointers separated by n steps; advance both until fast reaches end.

```
public ListNode removeNthFromEnd(ListNode head, int
    n) {
    ListNode dummy = new ListNode(0); dummy.next =
        head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i <= n; i++) fast = fast.next;
    while (fast != null) { fast = fast.next; slow =
        slow.next; }
    slow.next = slow.next.next;
    return dummy.next;
}
```

47. Reorder List

Time: O(N) — Space: O(1)

Q: Reorder list to: $L_0 \to L_n \to L_1 \to L_{n-1} \cdots$

Brute-force: Store node references in array/deque and re-link pointers ($O(N)$ space).

Optimal: Find middle (fast/slow pointers), reverse second half, merge two lists interleaved.

```
public void reorderList(ListNode head) {
    if (head == null) return;
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) { slow
        = slow.next; fast = fast.next.next; }
    ListNode prev = null, curr = slow.next;
        slow.next = null;
    while (curr != null) { ListNode nxt = curr.next;
        curr.next = prev; prev = curr; curr = nxt; }
    ListNode l1 = head, l2 = prev;
    while (l2 != null) {
        ListNode t1 = l1.next, t2 = l2.next;
        l1.next = l2; l2.next = t1; l1 = t1; l2 = t2;
    }
}
```

48. Longest Substring Without Repeating Characters

Time: O(N) — Space: O(N)

Q: Find length of longest substring without duplicates.

Brute-force: Check all $O(N^2)$ substrings for unique characters ($O(N^3)$ time).

Optimal: Sliding window with HashMap storing last seen indices of characters to shrink window left.

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> map = new HashMap<>();
    int max = 0, l = 0;
    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        if (map.containsKey(c)) l = Math.max(l,
            map.get(c) + 1);
        map.put(c, r);
        max = Math.max(max, r - l + 1);
    }
    return max;
}
```

49. Longest Repeating Character Replacement

Time: O(N) — Space: O(1)

Q: Longest substring of same letters achievable with k swaps.

Brute-force: Check all $O(N^2)$ substrings replacing up to k characters ($O(N^3)$ time).

Optimal: Sliding window tracking highest frequency count; shrink window when `len - maxFreq > k`.

```
public int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int l = 0, maxCount = 0, maxLen = 0;
    for (int r = 0; r < s.length(); r++) {
        maxCount = Math.max(maxCount,
            ++count[s.charAt(r) - 'A']);
        while (r - l + 1 - maxCount > k) {
            count[s.charAt(l) - 'A']--; l++; }
        maxLen = Math.max(maxLen, r - l + 1);
    }
    return maxLen;
}
```

50. Minimum Window Substring

Time: O(N) — Space: O(N)

Q: Find minimum window in s containing all characters of t.

Brute-force: Generate all $O(N^2)$ substrings and check if they contain all chars of t ($O(N^3)$ time).

Optimal: Sliding window tracking matched unique character frequencies.

```
public String minWindow(String s, String t) {
    int[] map = new int[128];
    for (char c : t.toCharArray()) map[c]++;
    int count = t.length(), l = 0, r = 0, minLen =
        Integer.MAX_VALUE, head = 0;
    while (r < s.length()) {
        if (map[s.charAt(r++)]-- > 0) count--;
        while (count == 0) {
            if (r - l < minLen) { minLen = r - l;
                head = l; }
            if (map[s.charAt(l++)]++ == 0) count++;
        }
    }
    return minLen == Integer.MAX_VALUE ? "" :
        s.substring(head, head + minLen);
}
```

51. Valid Anagram

Time: O(N) — Space: O(1)

Q: Check if s and t are anagrams.

Brute-force: Sort both strings and compare ($O(N \log N)$ time).

Optimal: Use an integer array of size 26 to count character frequencies in s and subtract for t.

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
            count[t.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

52. Group Anagrams

Time: O(N * K log K) — Space: O(N * K)

Q: Group array of strings into sublists of anagrams.

Brute-force: Compare every string pair by sorting/frequency matching ($O(N^2 \cdot K)$ time).

Optimal: Sort each string to form a canonical key; map key to its anagram group in a HashMap.

```
public List<List<String>> groupAnagrams(String[]
    strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] ca = s.toCharArray(); Arrays.sort(ca);
        String key = String.valueOf(ca);
        map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(s);
    }
    return new ArrayList<>(map.values());
}
```

53. Valid Parentheses

Time: O(N) — Space: O(N)

Q: Check if brackets close correctly.

Brute-force: Repeatedly replace empty pairs `()`, `{}`, `[]` ($O(N^2)$ string overhead).

Optimal: Push corresponding closing brackets onto Stack. Pop and match when encountered.

```
public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push(')');
        else if (c == '{') stack.push('}');
        else if (c == '[') stack.push(']');
        else if (stack.isEmpty() || stack.pop() !=
            c) return false;
    }
    return stack.isEmpty();
}
```

54. Valid Palindrome

Time: O(N) — Space: O(1)

Q: Check if string is palindrome ignoring non-alphanumeric.

Brute-force: Filter alphanumeric chars into new string, reverse it, and compare ($O(N)$ extra space).

Optimal: Use two pointers (left and right) moving inward, skipping non-alphanumeric elements.

```
public boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        while (l < r &&
            !Character.isLetterOrDigit(s.charAt(l)))
            l++;
        while (l < r &&
            !Character.isLetterOrDigit(s.charAt(r)))
            r--;
        if (Character.toLowerCase(s.charAt(l)) !=
            Character.toLowerCase(s.charAt(r)))
            return false;
        l++; r--;
    }
    return true;
}
```

55. Longest Palindromic Substring

```
public String longestPalindrome(String s) {
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expand(s, i, i), len2 = expand(s,
            i, i + 1);
        int len = Math.max(len1, len2);
        if (len > end - start) { start = i - (len -
            1) / 2; end = i + len / 2; }
    }
    return s.substring(start, end + 1);
}
private int expand(String s, int l, int r) {
    while (l >= 0 && r < s.length() && s.charAt(l)
        == s.charAt(r)) { l--; r++; }
    return r - l - 1;
}
```

56. Palindromic Substrings

```
public int countSubstrings(String s) {
    int count = 0;
    for (int i = 0; i < s.length(); i++) count +=
        expand(s, i, i) + expand(s, i, i + 1);
    return count;
}
private int expand(String s, int l, int r) {
    int res = 0;
    while (l >= 0 && r < s.length() && s.charAt(l)
        == s.charAt(r)) { res++; l--; r++; }
    return res;
}
```

57. Encode and Decode Strings

Time: O(N) — Space: O(1)

Q: Encode string list to single string and decode back.

Brute-force: Join strings with a special character like `","` (fails if words contain it).

Optimal: Prepend each word with length + delimiter (e.g., `"4#code"`) to avoid collisions.

```
public String encode(List<String> strs) {
    StringBuilder sb = new StringBuilder();
    for (String s : strs)
        sb.append(s.length()).append('#').append(s);
    return sb.toString();
}
public List<String> decode(String s) {
    List<String> res = new ArrayList<>(); int i = 0;
    while (i < s.length()) {
        int slash = s.indexOf('#', i);
        int len = Integer.parseInt(s.substring(i,
            slash));
        res.add(s.substring(slash + 1, slash + 1 +
            len));
        i = slash + 1 + len;
    }
    return res;
}
```

58. Max Depth Binary Tree

Time: O(N) — Space: O(H)

Q: Find height/depth from root to leaf node.

Brute-force: Traverse paths manually with explicit backtracks while accumulating path depth state.

Optimal: Post-order DFS: Return $1 + \max(depth(left), depth(right))$.

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left),
        maxDepth(root.right));
}
```

59. Same Tree

Time: O(N) — Space: O(H)

Q: Check if two binary trees are identical.

Brute-force: Serialize both trees completely into arrays/strings and compare array equality ($O(N)$ space).

Optimal: Recursive DFS: check `p.val == q.val` and compare corresponding subtrees concurrently.

```
public boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null || p.val != q.val)
        return false;
    return isSameTree(p.left, q.left) &&
        isSameTree(p.right, q.right);
}
```

60. Invert Binary Tree

Time: O(N) — Space: O(H)

Q: Mirror/swap left and right subtrees everywhere.

Brute-force: Construct an entirely new mirrored tree from scratch by duplicating nodes ($O(N)$ space).

Optimal: In-place traversal: recursively swap left and right pointers at every node.

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = invertTree(root.right); root.right =
        invertTree(temp);
    return root;
}
```

61. Binary Tree Max Path Sum

Time: O(N) — Space: O(H)

Q: Find max path sum between any two connected nodes.

Brute-force: Compute path sum between every pair of nodes independently using graph traversals ($O(N^3)$).

Optimal: DFS bottom-up: return maximum non-negative gain to parent; update global path max on the fly.

```
int max = Integer.MIN_VALUE;
public int maxPathSum(TreeNode root) { dfs(root);
    return max; }
private int dfs(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(0, dfs(node.left)), right =
        Math.max(0, dfs(node.right));
    max = Math.max(max, left + right + node.val);
    return node.val + Math.max(left, right);
}
```

62. Level Order Traversal

Time: O(N) — Space: O(N)

Q: Return level-by-level array representation.

Brute-force: DFS with depth indexing, inserting elements into variable-length arrays ($O(N \log N)$ sorting).

Optimal: BFS using FIFO Queue: process level sizes sequentially to cluster values into sublists.

```
public List<List<Integer>> levelOrder(TreeNode root)
    {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
        q.add(root);
    while (!q.isEmpty()) {
        int size = q.size(); List<Integer> level =
            new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();
                level.add(curr.val);
            if (curr.left != null) q.add(curr.left);
            if (curr.right != null)
                q.add(curr.right);
        }
        res.add(level);
    }
    return res;
}
```

63. Serialize/Deserialize Tree

Time: O(N) — Space: O(N)

Q: Convert binary tree to string and back.

Brute-force: Save full 2^H matrix representation of complete binary tree (wastes space on sparse trees).

Optimal: Pre-order traversal with delimiter (`","`) and marker for null nodes (`"X"`).

```
public String serialize(TreeNode root) {
    if (root == null) return "X,";
    return root.val + "," + serialize(root.left) +
        serialize(root.right);
}
public TreeNode deserialize(String data) {
    Queue<String> q = new
        LinkedList<>(Arrays.asList(data.split(",")));
    return buildTree(q);
}
private TreeNode buildTree(Queue<String> q) {
    String val = q.poll();
    if (val.equals("X")) return null;
    TreeNode node = new
        TreeNode(Integer.parseInt(val));
    node.left = buildTree(q); node.right =
        buildTree(q);
    return node;
}
```

64. Subtree of Another Tree

Time: O(N * M) — Space: O(H)

Q: Determine if `subRoot` is inside `root`.

Brute-force: Serialize both trees to string; run pattern search ($O(N + M)$ space for serialized strings).

Optimal: DFS on main tree: at each node, call an `isSameTree` helper matching with `subRoot`.

```
public boolean isSubtree(TreeNode root, TreeNode
    subRoot) {
    if (root == null) return false;
    if (isSame(root, subRoot)) return true;
    return isSubtree(root.left, subRoot) ||
        isSubtree(root.right, subRoot);
}
private boolean isSame(TreeNode s, TreeNode t) {
    if (s == null && t == null) return true;
    if (s == null || t == null || s.val != t.val)
        return false;
    return isSame(s.left, t.left) && isSame(s.right,
        t.right);
}
```

65. Construct Tree (Pre/Inorder)

Time: O(N) — Space: O(N)

Q: Reconstruct binary tree from order traversals.

Brute-force: Search linearly through inorder array on every recursive step to locate roots ($O(N^2)$).

Optimal: Preorder gives current root; use HashMap for $O(1)$ inorder splits between subtrees.

```
int preIdx = 0; Map<Integer, Integer> inMap = new
    HashMap<>();
public TreeNode buildTree(int[] preorder, int[]
    inorder) {
    for (int i = 0; i < inorder.length; i++)
        inMap.put(inorder[i], i);
    return helper(preorder, 0, inorder.length - 1);
}
private TreeNode helper(int[] pre, int inStart, int
    inEnd) {
    if (inStart > inEnd) return null;
    int rootVal = pre[preIdx++]; TreeNode root = new
        TreeNode(rootVal);
    int inIdx = inMap.get(rootVal);
    root.left = helper(pre, inStart, inIdx - 1);
    root.right = helper(pre, inIdx + 1, inEnd);
    return root;
}
```

66. Validate BST

Time: O(N) — Space: O(H)

Q: Check if binary tree follows valid BST ordering rules.

Brute-force: Compare each node only against its direct parent (misses deep illegal descendant values).

Optimal: DFS carrying upper and lower valid boundaries down to child nodes.

```
public boolean isValidBST(TreeNode root) { return
    validate(root, null, null); }
private boolean validate(TreeNode node, Integer min,
    Integer max) {
    if (node == null) return true;
    if ((min != null && node.val <= min) || (max !=
        null && node.val >= max)) return false;
    return validate(node.left, min, node.val) &&
        validate(node.right, node.val, max);
}
```

67. Kth Smallest Element in BST

Time: O(H + k) — Space: O(H)

Q: Find k-th smallest element in a BST.

Brute-force: Traverse all N nodes into a list, sort it, and fetch index $k - 1$ ($O(N \log N)$).

Optimal: In-order traversal visits BST elements in ascending order; stop counter when reaching k.

```
int count = 0, res = 0;
public int kthSmallest(TreeNode root, int k) {
    inorder(root, k); return res; }
private void inorder(TreeNode node, int k) {
    if (node == null) return;
    inorder(node.left, k);
    if (++count == k) { res = node.val; return; }
    inorder(node.right, k);
}
```

68. Lowest Common Ancestor (BST)

Time: O(H) — Space: O(1)

Q: Find lowest node sharing p and q as descendants.

Brute-force: Store full paths from root to p and q in lists and find divergence ($O(N)$ space).

Optimal: Iterate downwards: if both $p, q < root$, move left; if both $> root$, move right; else split node is LCA.

```
public TreeNode lowestCommonAncestor(TreeNode root,
    TreeNode p, TreeNode q) {
    while (root != null) {
        if (p.val < root.val && q.val < root.val)
            root = root.left;
        else if (p.val > root.val && q.val >
            root.val) root = root.right;
        else return root;
    }
    return null;
}
```

69. Implement Trie

Time: O(L) — Space: O(N * L)

Q: Build prefix tree with insert, search, and startsWith.

Brute-force: Store words in a List or HashSet; string search takes $O(N \cdot L)$ for prefix matching.

Optimal: Tree of nodes containing an array of 26 child references and a boolean `isEnd` flag.

```
class Trie {
    class Node { Node[] children = new Node[26];
        boolean isEnd; }
    private Node root = new Node();
    public void insert(String word) {
        Node curr = root;
        for (char c : word.toCharArray()) {
            if (curr.children[c - 'a'] == null)
                curr.children[c - 'a'] = new Node();
            curr = curr.children[c - 'a'];
        }
        curr.isEnd = true;
    }
    public boolean search(String word) { Node n =
        find(word); return n != null && n.isEnd; }
    public boolean startsWith(String pre) { return
        find(pre) != null; }
    private Node find(String s) {
        Node curr = root;
        for (char c : s.toCharArray()) {
            if (curr.children[c - 'a'] == null)
                return null;
            curr = curr.children[c - 'a'];
        }
        return curr;
    }
}
```

70. Add and Search Words

```
class WordDictionary {
    class Node { Node[] children = new Node[26];
        boolean isEnd; }
    private Node root = new Node();
    public void addWord(String word) {
        Node curr = root;
        for (char c : word.toCharArray()) {
            if (curr.children[c - 'a'] == null)
                curr.children[c - 'a'] = new Node();
            curr = curr.children[c - 'a'];
        }
        curr.isEnd = true;
    }
    public boolean search(String word) { return
        match(word, 0, root); }
    private boolean match(String w, int idx, Node
        node) {
        if (node == null) return false;
```

```
        if (idx == w.length()) return node.isEnd;
        char c = w.charAt(idx);
        if (c == '.') {
            for (Node ch : node.children) if (ch !=
                null && match(w, idx + 1, ch))
                return true;
            return false;
        } else return match(w, idx + 1,
            node.children[c - 'a']);
    }
}
```

71. Word Search II

```
public List<String> findWords(char[][] board,
    String[] words) {
    List<String> res = new ArrayList<>();
    TrieNode root = buildTrie(words);
    for (int r = 0; r < board.length; r++)
        for (int c = 0; c < board[0].length; c++)
            dfs(board, r, c, root, res);
    return res;
}
private void dfs(char[][] b, int r, int c, TrieNode
    p, List<String> res) {
    char ch = b[r][c];
    if (ch == '#' || p.next[ch - 'a'] == null)
        return;
    p = p.next[ch - 'a'];
    if (p.word != null) { res.add(p.word); p.word =
        null; }
    b[r][c] = '#';
    if (r > 0) dfs(b, r - 1, c, p, res);
    if (c > 0) dfs(b, r, c - 1, p, res);
    if (r < b.length - 1) dfs(b, r + 1, c, p, res);
    if (c < b[0].length - 1) dfs(b, r, c + 1, p,
        res);
    b[r][c] = ch;
}
class TrieNode { TrieNode[] next = new TrieNode[26];
    String word; }
private TrieNode buildTrie(String[] words) {
    TrieNode root = new TrieNode();
    for (String w : words) {
        TrieNode p = root;
        for (char c : w.toCharArray()) {
            if (p.next[c - 'a'] == null) p.next[c -
                'a'] = new TrieNode();
            p = p.next[c - 'a'];
        }
        p.word = w;
    }
    return root;
}
```

72. Merge k Sorted Lists

Time: O(N log k) — Space: O(k)

Q: Merge k sorted lists into a single sorted linked list.

Brute-force: Compare head nodes of all k lists linearly to pick minimum ($O(N \cdot k)$ time).

Optimal: Use Min-Heap storing head nodes of all non-empty lists to pick minimum in $O(\log k)$.

```
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return
        null;
    PriorityQueue<ListNode> pq = new
        PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode node : lists) if (node != null)
        pq.add(node);
    ListNode dummy = new ListNode(0), curr = dummy;
    while (!pq.isEmpty()) {
        ListNode top = pq.poll(); curr.next = top;
            curr = curr.next;
        if (top.next != null) pq.add(top.next);
    }
    return dummy.next;
}
```

73. Top K Frequent Elements

Time: O(N) — Space: O(N)

Q: Find k most frequent elements in array.

Brute-force: Count frequencies using HashMap, then sort entries by frequency ($O(N \log N)$).

Optimal: Map element frequencies, then use Bucket Sort array indexed by frequency counts.

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    for (int n : nums) count.put(n,
        count.getOrDefault(n, 0) + 1);
    List<Integer>[] bucket = new List[nums.length +
        1];
    for (int key : count.keySet()) {
        int freq = count.get(key);
        if (bucket[freq] == null) bucket[freq] = new
            ArrayList<>();
        bucket[freq].add(key);
    }
    int[] res = new int[k]; int idx = 0;
    for (int i = bucket.length - 1; i >= 0 && idx <
        k; i--) {
        if (bucket[i] != null) {
            for (int num : bucket[i]) {
                res[idx++] = num; if (idx == k)
                    break;
            }
        }
    }
    return res;
}
```

74. Median from Data Stream

Time: O(log N) add, O(1) median — Space: O(N)

Q: Design dynamic streaming median structure.

Brute-force: Maintain array of inputs sorted on every insert using insertion sort ($O(N)$ insertion).

Optimal: Maintain two heaps: Max-Heap for smaller numbers half, Min-Heap for larger numbers half.

```
class MedianFinder {
    private PriorityQueue<Integer> small = new
        PriorityQueue<>((a, b) -> b - a);
    private PriorityQueue<Integer> large = new
        PriorityQueue<>();
    public void addNum(int num) {
        small.add(num); large.add(small.poll());
        if (small.size() < large.size())
            small.add(large.poll());
    }
    public double findMedian() {
        return small.size() > large.size() ?
            small.peek() : (small.peek() +
            large.peek()) / 2.0;
    }
}
```

75. Task Scheduler

Time: O(N) — Space: O(1)

Q: Find minimum CPU intervals needed with cooling n.

Brute-force: Simulate interval steps using PriorityQueue and cooldown sub-queues ($O(N \log N)$ time).

Optimal: Greedy slot math: calculate idle gaps dictated strictly by the max frequency character.

```
public int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char t : tasks) freq[t - 'A']++;
    Arrays.sort(freq);
    int maxFreq = freq[25] - 1;
    int idleSlots = maxFreq * n;
    for (int i = 24; i >= 0 && freq[i] > 0; i--) {
        idleSlots -= Math.min(freq[i], maxFreq);
    }
    return idleSlots > 0 ? tasks.length + idleSlots
        : tasks.length;
}
```